

Guide Complet du Développeur - Plateforme Arcade

Ce guide exhaustif fusionne l'architecture globale, les détails de l'implémentation modulaire et les instructions techniques pour étendre la plateforme Arcade.

1. Architecture et Polymorphisme

Le projet repose sur une architecture modulaire utilisant le **polymorphisme**. Le programme principal (**le Core**) agit comme un chef d'orchestre : il ne connaît pas les implémentations concrètes (comment une bibliothèque dessine un pixel ou comment un jeu gère ses collisions), il interagit uniquement avec des interfaces abstraites : **IGraphics** et **IGame**.

Le cycle de vie (Game Loop)

Le Core exécute une boucle infinie qui suit ce schéma :

1. **Récupération des entrées** : Appel à `graphics->getInput()`.
2. **Mise à jour de la logique** : Transmission de l'entrée au jeu via `game->handleEvent(input)`, puis appel à `game->update()`.
3. **Rendu graphique** :
 - Nettoyage de l'écran : `graphics->clear()`.
 - Récupération des entités du jeu : `game->getEntities()`.
 - Rendu de chaque entité : `graphics->drawEntity(...)`.
 - Affichage final : `graphics->display()`.

2. Chargement Dynamique et Points d'entrée

Mécanisme Technique

Le Core utilise la bibliothèque système `<dlfcn.h>` (`dlopen`, `dlsym`, `dlclose`) pour interagir avec les bibliothèques partagées (`.so`).

- **Découverte** : Au démarrage, le Core scanne `./lib` à la recherche de fichiers `.so`.
- **Visibilité** : Pour permettre au noyau de trouver les fonctions de création sans "name mangling", chaque bibliothèque doit exporter ses symboles au format C.

Fonctions d'Usine (Factory Functions)

Chaque module doit implémenter les fonctions suivantes dans un bloc `extern "C"` :

Pour les modules Graphiques :

```
extern "C" {  
    Arcade::IGraphics* createGraphics() { return new MyLib(); }  
    void destroyGraphics(Arcade::IGraphics* lib) { delete lib; }  
}
```

Pour les modules de Jeux :

```
extern "C" {
    Arcade::IGame* createGame() { return new MyGame(); }
    void destroyGame(Arcade::IGame* game) { delete game; }
}
```

3. Interfaces et Dépendances

Structures de données partagées

Les échanges entre le jeu et la partie graphique se font via des structures normalisées :

- **Arcade::Color** : Énumération d'une palette de couleurs standard.
- **Arcade::GameEntity** : Représente un objet à afficher (position, symbole ASCII pour le mode texte, et chemin vers un sprite pour le mode graphique).
- **Arcade::GameText** : Structure pour l'affichage de texte (contenu, position, couleur, police).

L'Interface Graphique (IGraphics)

Toute bibliothèque graphique (SDL2, SFML, Ncurses, etc.) doit implémenter :

- `init()` / `close()` : Initialisation et nettoyage du contexte.
- `getInput()` : Lecture des entrées mappées sur `Arcade::Input`.
- `drawEntity(entity, colGrid, lineGrid)` : Affiche une entité en fonction de la taille de la grille de jeu.
- `drawText(text)` : Affiche du texte Formaté.

L'Interface Jeu (IGame)

Tout jeu (Snake, Pacman, etc.) doit implémenter :

- `init()` / `close()` : Initialisation (reset score, map) et nettoyage.
- `update()` : Calcul de la frame logique (mouvements, IA).
- `handleEvent(input)` : Réaction aux touches du joueur.
- `getEntities()` : Retourne la liste complète des objets visibles.

4. Guide d'Implémentation et Squelettes

Squelette minimal d'un Jeu (`src/games/MyGame.cpp`)

```
#include "games/IGame.hpp"

namespace Arcade {
    class MyGame : public IGame {
```

```

    public:
        void init() override { /* Reset score, player, etc. */ }
        void close() override { /* Cleanup */ }
        void update() override { /* Game logic here */ }
        void handleEvent(Input event) override { /* ... */ }
        std::vector<GameEntity> getEntities() const override { return
_entities; }
        int getScore() const override { return _score; }
        bool isGameOver() override { return _gameOver; }
    private:
        std::vector<GameEntity> _entities;
        int _score = 0;
        bool _gameOver = false;
};

extern "C" {
    Arcade::IGame *createGame() { return new Arcade::MyGame(); }
    void destroyGame(Arcade::IGame *game) { delete game; }
}

```

5. Build et Configuration

Flags de Compilation

Pour garantir la stabilité et la portabilité des bibliothèques partagées, utilisez les flags suivants :

- **-fPIC** : Indispensable pour créer une bibliothèque partagée.
- **-shared** : Indique au compilateur de générer un **.so**.
- **-fno-gnu-unique** : **CRITIQUE** sous Linux pour éviter que des symboles statiques ne persistent en mémoire après un **dlclose**.

Commande de compilation type

```

g++ -shared -fPIC -fno-gnu-unique -o lib/arcade_monjeu.so
src/games/MonJeu.cpp -I./include

```

6. Erreurs Courantes et Checklist

Pièges à éviter

[!WARNING] **Gestion de la Mémoire** : Ne détruisez jamais un module avec un simple **delete** dans le Core. Appelez toujours la fonction **destroyXXX** exportée par la bibliothèque.

[!CAUTION] **Exit Interdit** : Un module ne doit jamais appeler **exit()**. Il doit se contenter de signaler un état de fermeture au Core via ses méthodes d'interface.

Checklist de Validation Finale

- ☐ La classe hérite bien de l'interface **IGame** ou **IGraphics**.
- ☐ Les fonctions d'usine sont exportées via un bloc **extern "C"**.
- ☐ La bibliothèque est compilée avec **-fPIC** et **-fno-gnu-unique**.
- ☐ Le fichier de sortie respecte le format **lib/arcade_<nom>.so**.
- ☐ **getInput()** renvoie des valeurs mappées sur l'énumération **Arcade::Input**.